

Homework 06

```
# Load data and the provided train/test split (same split for every model)
df <- read.csv("~/Desktop/stat432-fall2026/week-06/data/breast-cancer.csv")
sp <- read.csv("~/Desktop/stat432-fall2026/week-06/data/breast-cancer-split.csv")
stopifnot(all(sp$row == seq_len(nrow(df))))
is_train <- sp$split == "train"

cols <- c("mean_radius", "mean_texture", "mean_smoothness")
Xtr <- as.matrix(df[is_train, cols]); ytr <- df$y[is_train]
Xte <- as.matrix(df[!is_train, cols]); yte <- df$y[!is_train]

mu <- colMeans(Xtr)
s <- apply(Xtr, 2, sd)
Xtr <- sweep(sweep(Xtr, 2, mu, "-"), 2, s, "/")
Xte <- sweep(sweep(Xte, 2, mu, "-"), 2, s, "/")

c(train = nrow(Xtr), test = nrow(Xte))
```

```
train  test
  398   171
```

```
# KNN estimate of P(y = 1): mean label of the k nearest training points
# (Euclidean distance, equal weights). When `query` is the training matrix,
# each point is its own nearest neighbor (distance 0).
knn_prob <- function(train, ytrain, query, k) {
  d2_train <- rowSums(train^2)
  d2_query <- rowSums(query^2)
  D2 <- outer(d2_query, d2_train, "+") - 2 * query %*% t(train)
  apply(D2, 1, function(d) mean(ytrain[order(d)[1:k]]))
}
```

```
# ROC curve and AUC (trapezoidal rule)
roc_curve <- function(y, p) {
  thr <- sort(unique(p), decreasing = TRUE)
  tpr <- c(0, sapply(thr, function(t) mean(p[y == 1] >= t)))
  fpr <- c(0, sapply(thr, function(t) mean(p[y == 0] >= t)))
  auc <- sum(diff(fpr) * (head(tpr, -1) + tail(tpr, -1)) / 2)
  list(fpr = fpr, tpr = tpr, auc = auc)
}
```

Question 1: Compare KNN and logistic regression

```
train_df <- data.frame(Xtr, y = ytr)
test_df <- data.frame(Xte)

# Ordinary logistic regression: intercept, no penalty
fit_lr <- glm(y ~ ., data = train_df, family = binomial)

probs <- list(Logistic = predict(fit_lr, newdata = test_df, type = "response"))
for (k in c(5, 15, 25, 35, 45)) {
  probs[[paste0("KNN k=", k)]] <- knn_prob(Xtr, ytr, Xte, k)
}
rocs <- lapply(probs, function(p) roc_curve(yte, p))
```

```
pal <- c("black", "#1b9e77", "#d95f02", "#7570b3", "#e7298a", "#66a61e")
plot(NA, xlim = c(0, 1), ylim = c(0, 1), xlab = "False positive rate",
     ylab = "True positive rate", main = "Test ROC curves")
abline(0, 1, lty = 2)
for (i in seq_along(rocs)) lines(rocs[[i]]$fpr, rocs[[i]]$tpr, col = pal[i], lwd = 2)
legend("bottomright", bty = "n", col = pal, lwd = 2,
     legend = sprintf("%s (AUC=%.3f)", names(rocs), sapply(rocs, `[[`, "auc")))
```

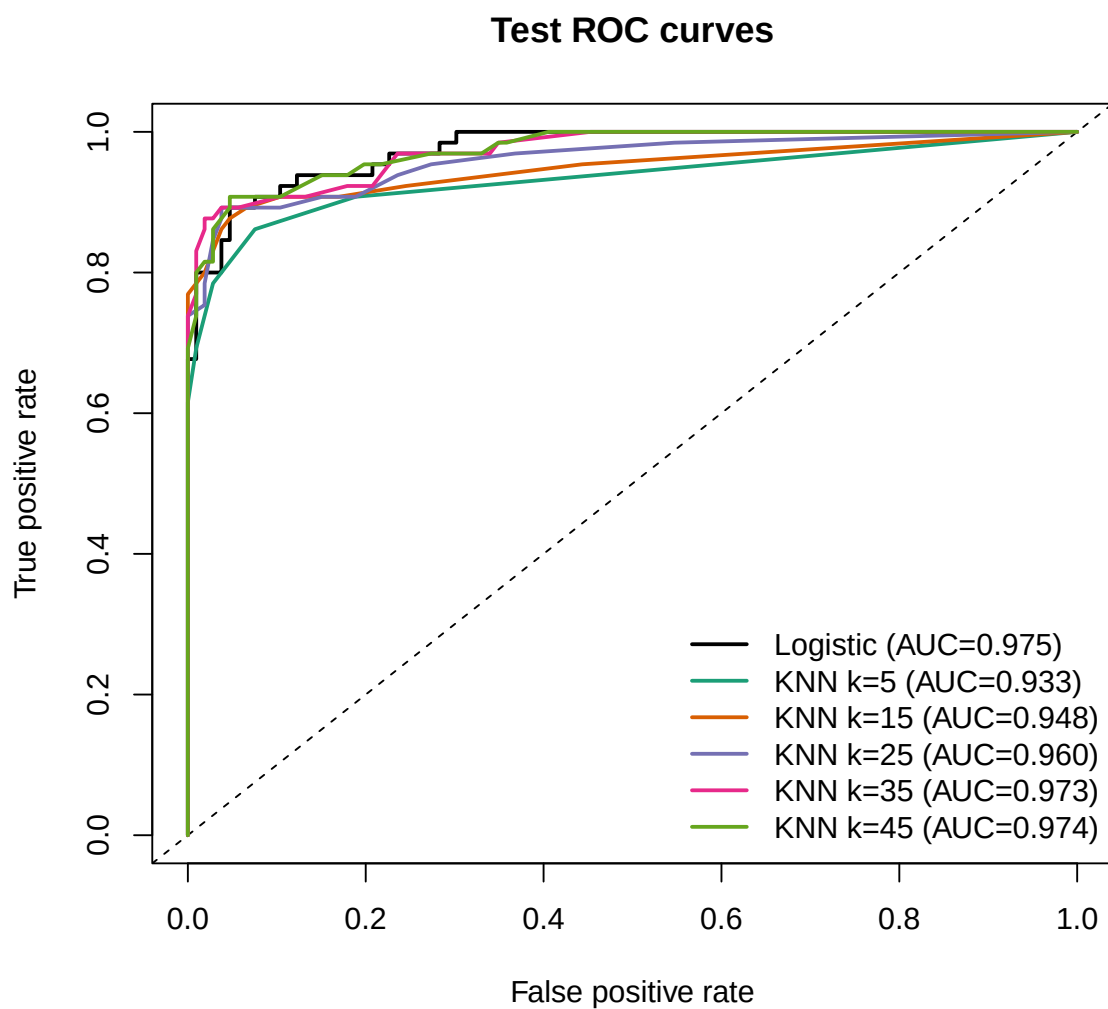


Figure 1: Test ROC curves for logistic regression and KNN.

```
knitr::kable(
  data.frame(Model = names(rocs), `Test AUC` = round(sapply(rocs, `[`, "auc"), 4),
    check.names = FALSE, row.names = NULL)
)
```

Model	Test AUC
Logistic	0.9746
KNN k=5	0.9329

Model	Test AUC
KNN k=15	0.9480
KNN k=25	0.9599
KNN k=35	0.9727
KNN k=45	0.9741

Preferred model: logistic regression. It has the highest test AUC (about 0.975). KNN improves steadily as k increases and reaches about 0.974 at $k = 45$, which is essentially tied with logistic regression. A gap that small is well within the noise for 171 test cases. Logistic regression is the simpler model (four parameters, no tuning parameter to choose), so I prefer it. KNN with small k ($k = 5$ and $k = 15$) is clearly worse, because averaging over only a few neighbors gives noisy probability estimates.

Question 2: Training and test ROC curves

```
ks <- c(1, 3, 5, 10, 20)
res <- data.frame(k = ks, train_auc = NA_real_, test_auc = NA_real_)

par(mfrow = c(1, 5), mar = c(4, 4, 3, 1))
for (i in seq_along(ks)) {
  k <- ks[i]
  r_tr <- roc_curve(ytr, knn_prob(Xtr, ytr, Xtr, k)) # point is its own neighbor
  r_te <- roc_curve(yte, knn_prob(Xtr, ytr, Xte, k))
  res$train_auc[i] <- r_tr$auc
  res$test_auc[i] <- r_te$auc

  plot(NA, xlim = c(0, 1), ylim = c(0, 1), xlab = "False positive rate",
       ylab = "True positive rate", main = paste("k =", k))
  abline(0, 1, lty = 2)
  lines(r_tr$fpr, r_tr$tp, col = "#1b9e77", lwd = 2)
  lines(r_te$fpr, r_te$tp, col = "#d95f02", lwd = 2)
  legend("bottomright", bty = "n", col = c("#1b9e77", "#d95f02"), lwd = 2,
       legend = c(sprintf("Train AUC=%.3f", r_tr$auc),
                   sprintf("Test AUC=%.3f", r_te$auc)))
}
```

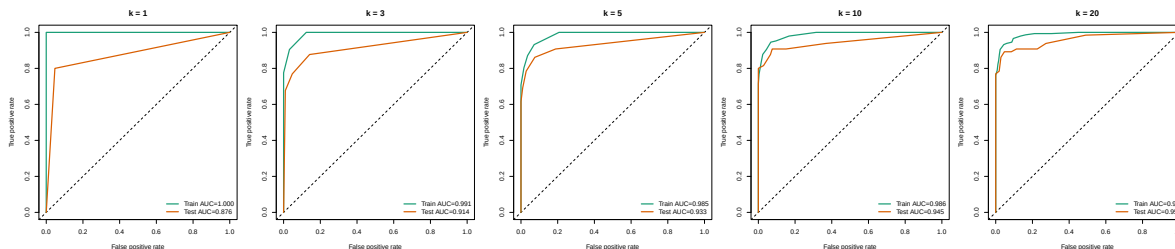


Figure 2: Training (green) and test (orange) ROC curves for KNN.

```
knitr::kable(
  data.frame(k = res$k,
    `Train AUC` = round(res$train_auc, 4),
    `Test AUC` = round(res$test_auc, 4),
    check.names = FALSE),
)
```

k	Train AUC	Test AUC
1	1.0000	0.8764
3	0.9905	0.9138
5	0.9847	0.9329
10	0.9862	0.9446
20	0.9880	0.9586

Training versus test. Training AUC is higher than test AUC for every k . The gap is largest at $k = 1$ (training AUC of 1.000 versus a test AUC of about 0.876) and shrinks as k grows, to about 0.03 at $k = 20$. Test AUC rises steadily with k , while training AUC falls from 1.0 and then stays roughly flat at about 0.985 to 0.99.

What happens at $k = 1$. When a training observation is used to predict itself, it is its own nearest neighbor at distance 0. Its estimated probability is therefore exactly its own label, 0 or 1, so the training ROC curve goes straight to the top-left corner and the AUC equals 1. This is true regardless of how well the model would predict new cases. (It could fail only if duplicate predictor values carried conflicting labels.)

Why test performance is lower. A new observation has no copy of itself in the training data. At $k = 1$ it inherits the label of a single nearby point, which may reflect noise rather than signal. The estimated probabilities are only 0 or 1, so the test ROC curve has just one corner point, and its AUC is the lowest of all the models.

How performance changes with k . Larger k averages the labels of more neighbors. This reduces variance, smooths the decision boundary and produces graded probabilities. Test

AUC improves, and the training and test curves move closer together. The model moves from overfitting toward a smoother fit.

Why training performance is misleading. The model has already seen the training labels, and at small k it effectively memorizes them. Training AUC would always select the most flexible model ($k = 1$) as best, so it cannot be used to choose k or to compare models. An honest estimate of performance on new patients requires held-out data or cross-validation.

Question 3: Peer evaluation of Homework 5

Peer Review for Hilm Chuang

1. Mistake or item for improvement

In hw-05, Question 2a/2b, the KNN vs. Lasso comparison could be made fairer. Lasso's is tuned by cross-validation, while KNN uses a fixed $k = 5$, and the conclusion comes from a single simulation.

2. Assessment

The code fits KNN with `knn.reg(X, test.X, y, k = 5, algorithm = "brute")`, while Lasso gets a full CV search over `.`. Since KNN's performance depends a lot on k , giving it the same tuning would make the comparison more convincing. The 2b text says KNN performs "slightly better" in the correlated setting, but 0.161 vs. 0.226 is a gap of roughly 29%, which is a bit more than slight. With just one simulation, it's also hard to tell whether the ordering would hold on a repeat.

3. Correction

A fairer approach would be to tune k by 10-fold CV over `seq(1, 29, 2)` using the same folds as Lasso, refit with the best k , and repeat the whole procedure (say, 50 times) to report the mean and SD of each test MSE. The 2b explanation could then read: "With k tuned, KNN outperforms Lasso in the correlated setting. Correlated features lie near a low-dimensional region, so neighbors are close in the relevant coordinates, while Lasso cannot capture the nonlinear terms. When features are independent, Lasso does better, because the irrelevant features dilute the distances KNN relies on."

GitHub Issue

<https://github.com/HiImChuang/stat432-fall2026/issues/3>

Peer Review for Mann Talati

1. Mistake or item for improvement

hw-05, Question 3c (role of ϵ): the explanation of how the noise affects neighbor ranking is slightly off.

2. Assessment

The text says the noise adds about $2 \times 100 = 200$ to the expected squared distance between any two points “regardless of how similar their latent variables are, which blurs the ranking of neighbors.” The 200 is right, but a constant added to every pair cannot change which points rank as nearest. What blurs the ranking is the random fluctuation of the noise term around 200, which the answer never mentions.

3. Correction

Each squared coordinate difference of ϵ has variance 8, so over 100 coordinates the noise part of the squared distance has variance 800, a standard deviation of about 28. Suggested rewrite: “The noise adds about 200 to every squared distance on average, which does not change the ranking by itself. What matters is that it fluctuates randomly (SD about 28), so a point with a slightly smaller latent distance can be outranked by one with luckier noise. That is why the nearest neighbors in X are only approximately the nearest in Z .”

GitHub Issue

<https://github.com/manntalati/stat432-fall2026/issues/2>